



UEA

UNIVERSIDAD
ESTATAL AMAZÓNICA



UEA
UNIVERSIDAD
ESTATAL AMAZÓNICA

ASIGNATURA

FUNDAMENTOS DE PROGRAMACIÓN

Unidad 3

Arreglos N-Dimensionales

Tema 3.2.

Operaciones con arreglos multidimensionales

Subtema 3.2.1. Búsqueda y ordenación de elementos en arreglos multidimensionales.



UEA

UNIVERSIDAD
ESTATAL AMAZÓNICA

**FUNDAMENTOS DE PROGRAMACIÓN
(UEA-L-UFB-026)**

Arreglos Multidimensionales

- Un arreglo multidimensional es una estructura de datos que organiza elementos en múltiples dimensiones.
- Estos arreglos permiten almacenar datos de forma estructurada. En Java son esenciales para manejar tablas, matrices y datos complejos en memoria.

```
#Python  
mi_arreglo = [  
    [1, 2, 3],  
    [4, 5, 6]  
]
```

Importancia de la Búsqueda y Ordenación

- Cuando trabajamos con arreglos multidimensionales, a menudo necesitamos buscar elementos específicos o realizar algún tipo de organización de los datos para facilitar su acceso y análisis.

```
# Búsqueda de un valor específico
```

```
valor_buscado = 3  
encontrado = False
```

```
for fila in matriz:  
    for v in fila:  
        if v == valor_buscado:  
            encontrado = True
```

```
if encontrado:  
    print(f"Se encontró {valor_buscado} en la matriz.")  
else:  
    print(f"{valor_buscado} no se encontró en la matriz.")
```

```
# Ordenar cada fila de la matriz
```

```
for fila in matriz:  
    fila.sort()
```

Importancia de la Búsqueda y Ordenación

- La búsqueda y ordenación son habilidades esenciales que se aplican en numerosos escenarios de programación, desde procesar datos en una hoja de cálculo hasta buscar registros en una base de datos.

```
# Matriz de ejemplo
```

```
matriz = [
```

```
    [5, 2, 8],
```

```
    [1, 9, 3],
```

```
    [4, 7, 6]
```

```
]
```

```
# Búsqueda de un valor específico
```

```
valor_buscado = 3
```

```
encontrado = False
```

```
for fila in matriz:
```

```
    for v in fila:
```

```
        if v == valor_buscado:
```

```
            encontrado = True
```

```
if encontrado:
```

```
    print("Se encontró", valor_buscado, "en la matriz.")
```

```
else:
```

```
    print(valor_buscado, "no se encontró en la matriz.")
```

```
# Ordenar cada fila de la matriz
```

```
for fila in matriz:
```

```
    fila.sort()
```

```
print("Matriz ordenada por filas:")
```

```
print(matriz)
```

Búsqueda en Arreglos Multidimensionales

- La búsqueda implica encontrar un elemento específico dentro de un arreglo multidimensional.
- Existen diferentes algoritmos de búsqueda, como la búsqueda lineal y la búsqueda binaria, que se pueden aplicar en estas estructuras de datos.
- La elección del algoritmo depende de factores como la cantidad de datos y la eficiencia requerida.

Búsqueda Lineal en Arreglos Multidimensionales

- La búsqueda lineal es uno de los métodos más simples de búsqueda en arreglos multidimensionales.
- En esta búsqueda, comenzamos en el primer elemento y avanzamos uno por uno hasta encontrar el elemento deseado o llegar al final del arreglo.
- En un arreglo bidimensional, la búsqueda lineal se realiza recorriendo primero las filas y luego las columnas (o viceversa), verificando cada elemento en busca del valor deseado.
- La búsqueda lineal es efectiva en arreglos no ordenados o cuando no se conoce la estructura de los datos.
- Es importante tener en cuenta que la búsqueda lineal puede ser menos eficiente en arreglos grandes, ya que debe revisar cada elemento en el peor de los casos.

Ejemplo Python

Este es un ejemplo básico de búsqueda lineal en una matriz bidimensional. La búsqueda lineal es útil cuando no se conoce la estructura de los datos o cuando los datos no están necesariamente ordenados.

```
# Crear una matriz bidimensional (3x3)
matriz = [
    [4, 7, 2],
    [1, 9, 6],
    [8, 3, 5]
]
# Valor que estamos buscando
valor_buscado = 9
# Variables para rastrear la posición
fila_encontrada = -1
columna_encontrada = -1
# Recorrer la matriz para buscar el valor
for i in range(len(matriz)):
    for j in range(len(matriz[i])):
        if matriz[i][j] == valor_buscado:
            fila_encontrada = i
            columna_encontrada = j
            break
# Detener también el ciclo externo
if fila_encontrada != -1:
    break
```

Búsqueda Binaria en Arreglos Bidimensionales:

- La búsqueda binaria es un método más eficiente cuando trabajamos con arreglos bidimensionales que están ordenados.
- En lugar de buscar elemento por elemento, la búsqueda binaria divide el espacio de búsqueda en dos partes y elimina una de ellas en cada paso.
- Para realizar una búsqueda binaria en un arreglo bidimensional, primero se selecciona una fila o columna para dividir.
- Supongamos que ordenamos las filas de manera ascendente y las columnas de manera descendente.
- Comenzamos en una esquina de la matriz y comparamos el valor deseado con el valor en esa posición.

Búsqueda Binaria en Arreglos Bidimensionales:

- Si el valor deseado es igual al valor en esa posición, hemos encontrado el elemento.
- Si el valor deseado es menor, podemos eliminar la fila actual y buscar en la mitad superior de la matriz.
- Si el valor deseado es mayor, podemos eliminar la columna actual y buscar en la mitad derecha de la matriz.
- Repetimos este proceso hasta encontrar el elemento o determinar que no está en la matriz.
- La búsqueda binaria es altamente eficiente en arreglos ordenados, ya que reduce significativamente el número de comparaciones necesarias para encontrar un elemento.

En Resumen

- Esta diapositiva introduce dos enfoques de búsqueda:
 1. La búsqueda lineal, que es simple pero puede ser menos eficiente en arreglos grandes o desordenados.
 2. La búsqueda binaria, que es altamente eficiente en arreglos ordenados.

La elección entre estos métodos dependerá de la estructura y las condiciones de los datos en el arreglo multidimensional.

Algoritmos de Ordenación en Arreglos Multidimensionales

Los algoritmos de ordenación son fundamentales en la programación y son utilizados para organizar elementos en un arreglo en un orden específico, ya sea de manera ascendente o descendente.

Método de la
Burbuja (Bubble
Sort)



QuickSort



MergeSort

Método de la Burbuja (Bubble Sort)

- El método de la burbuja es uno de los algoritmos de ordenación más simples y fáciles de entender.
- Funciona comparando pares de elementos adyacentes y realizando intercambios si están en el orden incorrecto.
- El proceso se repite hasta que no se realizan intercambios en un pase completo a través del arreglo.
- Aunque es fácil de implementar, el método de la burbuja no es eficiente para arreglos grandes debido a su complejidad de tiempo cuadrática.

QuickSort

- QuickSort es un algoritmo de ordenación rápido y eficiente que se basa en la estrategia de "divide y vencerás".
- Selecciona un elemento pivote y divide el arreglo en dos subarreglos: uno con elementos menores que el pivote y otro con elementos mayores.
- Luego, ordena recursivamente los subarreglos.
- QuickSort es conocido por su eficiencia en arreglos grandes y se utiliza comúnmente en la práctica.

MergeSort

- MergeSort es otro algoritmo de ordenación basado en "divide y vencerás".
- Divide el arreglo en dos mitades, ordena cada mitad y luego combina las dos mitades ordenadas en un solo arreglo ordenado.
- Aunque MergeSort es más eficiente que el método de la burbuja, su complejidad de tiempo es mayor que la de QuickSort.
- Sin embargo, MergeSort es estable y eficiente en la ordenación de arreglos multidimensionales.

Elección del Algoritmo de Ordenación

- La elección del algoritmo de ordenación depende de varios factores, incluida la cantidad de datos, la estructura de los datos y los recursos disponibles.
- Es importante considerar la eficiencia y la estabilidad de los algoritmos al seleccionar uno para un proyecto específico.
- En algunos casos, es posible que se prefiera un algoritmo de ordenación específico para un tipo particular de datos o estructura multidimensional.

Importancia de la Ordenación

- La ordenación es esencial en la programación para organizar datos y facilitar su búsqueda y análisis.
- Los algoritmos de ordenación juegan un papel clave en aplicaciones como bases de datos, procesamiento de imágenes, análisis de datos y más.
- Comprender y aplicar algoritmos de ordenación es una habilidad valiosa para cualquier programador.

***JUNTOS TRANSFORMAMOS
EL MUNDO DESDE
LA AMAZONÍA***

